



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目 基于智能体的漏洞挖掘与修复技术研究

学生姓名 张小北 学号 202518018629028

指导教师 侯锐 职称 研究员

学位类别 工学博士

学科专业 计算机系统结构

研究方向 漏洞治理

培养单位 中国科学院信息工程研究所

填表日期 2026 年 09 月 28 日

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	2
二 国内外本学科领域的发展现状与趋势	4
2.1 基于智能体的漏洞挖掘	4
2.2 基于智能体的漏洞自动修复	8
2.3 基于智能体的漏洞修复验证	12
三 课题主要研究内容与预期目标	16
3.1 研究目标	16
3.2 研究内容一：基于智能体的漏洞挖掘方法	17
3.3 研究内容二：基于智能体的漏洞自动修复方法	17
3.4 研究内容三：基于智能体的漏洞修复验证方法	18
四 拟采用的研究方法、技术路线、实验方案及其可行性分析 ..	19
4.1 基于智能体的漏洞挖掘方法	19
4.2 基于智能体的漏洞自动修复方法	21
4.3 基于智能体的漏洞修复验证方法	24
五 已有科研基础与所需的科研条件	27
5.1 已有科研基础	27
5.2 所需科研条件	28
六 研究工作计划与进度安排	29

一 选题的背景及意义

1.1 选题背景

安全漏洞是系统中可被威胁源利用或触发、导致安全策略被违反的缺陷或弱点，可能影响信息的保密性、完整性或系统的可用性 [1]。本课题主要关注软件与固件实现中的安全漏洞。与一般功能缺陷不同，漏洞分析不仅需要发现异常行为，还需要进一步确定漏洞的触发条件、影响范围以及可能造成的安全后果。因此，程序是否产生崩溃并不能作为判断漏洞存在的唯一依据，许多协议逻辑错误、状态管理错误和资源生命周期问题往往需要结合系统行为和安全约束进行分析。

漏洞治理是一个贯穿漏洞生命周期的系统性过程，并不仅限于漏洞发现或补丁发布，而是包括漏洞发现与报告、复现确认、根因分析、影响评估、修复生成、补丁验证以及部署后的修复状态核查等多个环节 [2-4]。其中，漏洞挖掘回答“系统中是否存在安全缺陷”，漏洞修复回答“如何消除该缺陷”，而修复验证则进一步确认“生成的修复是否真正有效以及是否已经应用到实际运行环境”。因此，漏洞治理的自动化不仅需要提升单个环节的分析能力，还需要建立贯穿发现、修复和验证全过程的可靠技术体系。

传统漏洞治理方法主要依赖静态分析、符号执行、模糊测试以及基于规则的程序修复技术，在内存破坏等具有明确错误信号的问题上取得了较好的效果。然而，对于复杂软件系统中的语义缺陷和跨组件问题，现有方法仍面临程序语义理解不足、测试预言难以构造以及修复正确性难以判断等挑战。例如，嵌入式设备和剥离符号二进制场景中缺少完整源码和运行信息，使漏洞定位与行为理解更加困难；漏洞自动修复过程中，检测位置与实际根因位置可能存在偏差；已有测试通过也不能充分证明补丁消除了漏洞根因。

近年来，大语言模型（Large Language Model, LLM）和智能体（Agent）技术的发展，为漏洞治理提供了新的研究范式。相比传统依赖固定规则和人工流程的分析方法，基于智能体的方法能够结合代码理解、工具调用和执行反馈，主动完成程序分析、测试生成、漏洞定位以及补丁修改等多步骤任务。Google Big Sleep 已展示基于智能体在真实开源项目中发现未知漏洞的能力，PatchAgent 则进一步探索了利用智能体完成漏洞定位、补丁生成和验证的自动化流程 [5, 6]。这些研

究表明，智能体具备连接漏洞语义理解与程序操作执行的潜力，为构建自动化漏洞治理体系提供了新的技术路径。

然而，智能体技术的发展并未完全解决漏洞治理中的核心问题。在漏洞挖掘方面，LLM 可以利用自然语言理解能力分析协议规范、接口说明和程序逻辑，为复杂漏洞测试提供语义指导；但对于仅能获取设备或剥离符号二进制的场景，程序内部状态、行为约束和执行环境仍难以准确恢复 [7, 8]。同时，协议状态违例、行为逻辑错误和资源生命周期问题通常缺少明确的崩溃信号，需要结合系统规约和安全性质构造有效的判定机制 [9]。

在漏洞修复方面，LLM 已经展现出根据漏洞描述和代码上下文生成补丁的能力，但真实漏洞修复仍面临根因定位不准确、上下文信息不足以及补丁过拟合等问题。漏洞触发位置并不一定对应实际修复位置，跨函数和跨模块依赖关系也增加了根因分析难度 [10]。同时，模型可能根据已有漏洞模式生成局部合理但不完整的修复方案，导致漏洞触发被抑制但安全性质未真正恢复 [11]。

在漏洞修复验证方面，智能体可以辅助构造测试和分析程序行为，但如何独立判断补丁是否真正消除了漏洞仍然缺少有效方法。已有研究表明，通过公开 PoC 和功能测试并不能充分证明补丁正确，补丁可能仅针对已知输入进行适配，而未解决漏洞根因 [12]；此外，软件供应链中发布补丁也不意味着所有实际部署的软件版本均已完成修复，需要进一步确认目标程序是否包含对应安全修改 [4]。

基于上述背景，本课题以“基于智能体的漏洞挖掘与修复技术研究”为题，围绕“发现问题—生成修复—确认修复”的漏洞治理主线，开展**基于智能体的漏洞挖掘、基于智能体的漏洞自动修复以及基于智能体的漏洞修复验证**三个方向的研究，分别面向复杂场景中的漏洞发现、根因驱动的补丁生成以及修复有效性和目标修复状态确认，提升漏洞治理关键环节的自动化水平与可靠性。

1.2 选题意义

在研究层面，本课题围绕安全要求与程序行为之间的对应关系，探索大模型语义推理与程序分析、动态测试相结合的方法。通过研究行为约束如何支持漏洞判定、根因理解如何指导有效修改，以及何种程序证据能够支持修复结论，有助于深化对不完备信息条件下智能体安全分析能力及其边界的认识。同时，区分漏洞发现、修复有效性与目标修复状态的评价依据，可为相关方法的设计和比较提

供更明确的标准。

在应用层面，本课题有望提高嵌入式设备与二进制组件中隐蔽安全缺陷的发现能力，降低复杂漏洞定位与补丁编写的人工成本。通过同时关注安全要求与合法行为，减少不完整修复和功能回归；通过核查目标产物中的修复实现，帮助维护者识别补丁发布与实际修复状态之间的差异。相关成果可为软件安全维护、嵌入式设备审计和第三方组件更新核查提供技术支撑。

二 国内外本学科领域的发展现状与趋势

2.1 基于智能体的漏洞挖掘

2.1.1 研究背景

漏洞挖掘旨在识别目标系统中违反安全要求的程序行为，并提供支持缺陷判定与复现的证据。静态分析、符号执行和模糊测试分别从程序结构、路径约束和实际执行出发开展检测，形成相互补充的技术体系：覆盖率引导的模糊测试持续生成输入并保留有探索价值的测试用例，显著提高了自动化发现缺陷的效率；AddressSanitizer 等运行时检测机制为越界访问、释放后使用等错误提供了可复用的判据 [13, 14]。这一体系的有效运行，还依赖目标可执行性、关键状态可达性以及错误是否具有可观察的判定依据。

大语言模型和智能体的发展正在降低代码理解、测试程序编写及分析工具编排的人工成本。Google 的 Big Sleep 已通过代码分析与工具交互，在经过长期测试的 SQLite 中发现未知内存安全漏洞；Mozilla 于 2026 年 4 月报告，使用前沿模型在初步评估中发现的 271 项漏洞已随 Firefox 150 修复 [5, 15]。这些进展说明智能体已能参与成熟项目的实际安全分析，也表明内存安全问题尚未被普遍解决；在长期充分测试的特定组件中，仅继续增加同类输入变异的边际收益正在下降，新的增量能力更加依赖对深层程序语义与测试盲区的理解。因此，后续研究需要考察自动化能力尚未充分覆盖的任务条件。

相比能够通过崩溃、异常或崩溃报告直接暴露的内存错误，许多安全缺陷并不存在统一、显式的错误信号。例如，协议实现可能违反状态转移要求，权限状态可能在错误的会话间延续，资源也可能在生命周期结束后持续占用，而程序仍能正常执行。此类问题的关键不再只是能否触发新的执行路径，而是能否获得相应的安全要求，并据此判断实际行为是否构成违例，即测试预言（test oracle）问题 [9]。因此，本文重点关注协议与状态语义、访问控制和资源生命周期等需要结合行为约束才能判定的安全缺陷。

上述问题在嵌入式与固件场景中进一步受到分析条件的限制：仅能与设备交互时，内部状态、代码覆盖及资源对象通常不可直接观察；剥离符号的二进制需要恢复对象、功能与程序位置之间的对应关系；固件重托管后的行为也可能依赖外设响应、配置、定时事件与跨组件交互。Firmalice 和 Fuzzware 分别从二进

制安全语义分析与硬件接口建模出发处理其中的部分困难，说明这些限制需要专门的程序分析与执行支持 [8, 16]。

因此，本课题关注在**源码、符号、规约或运行时观察不完备的条件下，如何利用大语言模型的语义理解与交互分析能力，发现违反行为约束且能够在真实环境中成立的安全缺陷**。研究重点由单纯增加输入与路径覆盖，转向安全要求的获得、目标行为的辨识、状态违例的触发及其可信判定。

2.1.2 研究现状与趋势

传统漏洞挖掘主要依赖静态分析、符号执行和模糊测试，并逐渐从代码路径覆盖扩展到程序状态和安全性质。AFLNet、StateAFL 等工作利用协议响应或运行时状态反馈增强有状态程序的路径探索 [17, 18]；LTL-Fuzzer 等进一步将显式行为性质作为测试目标，搜索违反时序约束的执行轨迹 [19]。这些方法提高了复杂状态空间中的探索能力，但通常需要预先定义状态表示、安全性质及其程序事件映射。对于缺少完整规约和语义信息的目标，如何获得可信的安全要求并将其转化为可执行的测试判据仍是重要问题。

大模型的发展进一步增强了漏洞挖掘中的输入生成和分析能力。Fuzz4All 利用（Large Language Model, LLM）生成和变异测试输入，ChatAFL 则利用模型的协议知识生成消息并探索新的协议状态 [20, 21]。近期研究进一步将模型置于持续的工具交互过程中，使其参与代码理解、假设生成、测试和证据分析。相比传统模糊测试，这类方法显著增强了输入构造和分析过程的自动化，但“能够探索更多行为”并不意味着“能够正确判断安全违例”，安全假设和测试 oracle 的可靠性仍需独立解决。

这一问题在缺少源码和符号信息的二进制与固件场景中更加突出。BINREX 结合语义检索、分层规划和 IDAPython 工具调用实现任务驱动的二进制分析 [7]；FirmAgent 则结合模糊测试与 LLM 污点分析，在固件中建立实际输入与潜在危险路径之间的联系 [22]。与此同时，Fuzzware 等工作通过建模 MMIO 等外部环境提高固件动态执行能力 [8]。这些研究改善了语义恢复、路径定位和固件可执行性，但对于依赖历史状态、环境变化和多步交互的漏洞，仅恢复代码语义或提高执行覆盖仍不足以形成可靠的安全判据。

因此，近期研究开始进一步利用协议规范和历史缺陷知识提供安全语义。

BSFuzzer 利用大语言模型从蓝牙协议规范中恢复状态机和报文语义，并据此构造字段和状态转移违例；ProtocolGuard 从协议规范提取约束，将其映射到源码并生成可执行断言进行验证 [23, 24]。另一类工作则从历史修复中提取可复用的行为约束，例如 TyPatch 利用 LLM 从 Linux 补丁中提取对象、动作、守卫和状态转移等类型状态转移规则 [25]。这些工作说明，规约和历史补丁都可以为漏洞挖掘提供比代码覆盖更明确的安全要求，但现有方法通常依赖较完整的规范、源码或静态分析条件，如何在信息受限的二进制和设备环境中恢复并验证这些约束仍有困难。

此外，大量逻辑和资源缺陷并不会直接产生崩溃，使漏洞挖掘进一步面临“如何判错”的问题。LISA 通过生成 API 调用序列和程序不变式发现功能异常 [26]，InferROI 则利用 LLM 推断资源获取、释放和可达性意图，并结合静态分析检测资源泄漏 [27]。这些工作表明，漏洞检测可以利用崩溃之外的行为和资源性质，但对于长期资源占用、状态错误和协议逻辑异常，仅观察执行结果、资源曲线或对象可达性通常不足以判断是否真正违反安全要求，还需要结合对象生命周期、历史状态和预期行为进行解释。

综合来看，漏洞挖掘正在由单纯追求输入和路径覆盖，逐步转向基于状态、规约和安全性质理解程序行为；大模型与智能体的发展又进一步增强了语义恢复、工具使用和测试生成能力。然而，现有能力仍主要分散在状态探索、规约提取、二进制分析、环境建模和定制 oracle 等不同环节。进一步需要解决的核心问题是：**在源码、规约和执行环境均可能不完整的条件下，如何获得可信的安全要求，将其映射为目标程序可观察、可执行的行为条件，并形成真实可达、可复现的漏洞证据。**

2.1.3 研究挑战

首先，**行为规约的可信获得**是行为约束驱动漏洞挖掘的基础。协议规范、源码和历史补丁分别描述预期行为、实际实现和历史修复条件，其完整性、适用范围和可信度并不一致。已有研究表明，规约解释或上下文缺失会直接影响后续检测结果 [24, 25]。因此，在缺少完整规范时，需要区分明确规定的安全义务、具有独立证据支持的推断和待验证假设，避免直接将程序中的常见行为或模型生成的解释作为安全规范。

其次，需要解决**安全概念与目标程序行为之间的映射**。在缺少源码和符号信息的场景中，即使能够恢复函数语义，也仍需确定安全要求具体对应哪个对象、状态和程序位置，以及状态变化如何影响后续行为。BINREX 等工作增强了剥离二进制中的语义恢复能力 [7]，但对于跨函数、跨任务和多组件共同维护的状态，仅识别函数功能仍不足以判断安全约束是否被违反。设备黑盒场景下内部状态进一步不可见，同一响应也可能对应不同内部状态，因此需要根据源码、二进制和设备等不同可观察条件建立不同层次的行为证据。

进一步地，状态性漏洞要求构造**真实可达的违例轨迹**。协议状态、资源生命周期和硬件环境通常由多次调用及外部事件共同决定，已有状态感知模糊测试能够增强多步交互探索 [18, 21]，但生成任意非法操作往往只会触发早期拒绝，并不能到达真正需要检查的状态。在重托管和模拟执行环境中，还必须区分人为构造的内部状态与攻击者能够通过合法外部操作到达的状态。只有能够说明目标状态的到达过程及其环境假设，异常行为才能形成具有实际意义的漏洞证据。

另一个核心问题是**缺少直接错误信号时的语义判定**。逻辑错误、协议违例和资源泄漏通常不会产生崩溃，需要通过不变式、断言或定制 oracle 判断行为是否违反安全要求 [24, 27]。这类判据既不能过弱而遗漏漏洞，也不能过强而拒绝合法行为；资源类缺陷还涉及生命周期和观察窗口，例如未立即释放可能来自异步回收，资源增长也可能来自合法缓存。因此，需要同时定义预期安全行为和合法对照行为，并将候选发现与独立验证分离，使最终结论能够被重复执行和反驳。

最后，智能体化漏洞挖掘还需要从单点能力提升转向**完整发现过程的可靠评价**。代码覆盖、状态覆盖、性质监测状态和资源消耗都只能反映分析过程的部分信息，代理指标的提升并不必然意味着发现了更多真实漏洞。评价时还需控制源码、规约、历史补丁等输入条件，区分静态候选、可触发缺陷和经确认漏洞，并记录工具执行、环境构建和人工确认等成本。只有保持输入条件和证据标准一致，才能判断性能提升究竟来自模型能力、分析方法还是额外信息。

综上，智能体与大模型增强了语义理解、状态探索 and 工具使用能力，但并未消除规约缺失、状态不可见和非崩溃行为难以判定等根本问题。面向嵌入式设备和剥离二进制，本课题需要进一步建立**可信行为约束、真实可达交互与可靠语义判定之间的联系**，并以此连接行为规约、状态探索、可达性分析和漏洞判定。

2.2 基于智能体的漏洞自动修复

2.2.1 研究背景

漏洞修复是软件安全维护的重要环节，其目标除阻止已知攻击输入触发异常外，还需消除导致危险行为的程序缺陷，并维持正常使用所依赖的功能与安全约定。漏洞自动修复（Automated Vulnerability Repair, AVR）将漏洞分析、补丁生成与补丁验证组织为自动化过程，以降低安全维护对人工分析的依赖。与面向一般软件缺陷的自动程序修复（Automated Program Repair, APR）相比，AVR 更强调在攻击者可控输入、异常状态及安全敏感操作下恢复必要的安全条件。修复目标需要同时考察危险行为是否被有效约束、相关合法行为是否得到保留，代码修改是否合理、程序能否编译或已知失败是否消失只能作为局部指标 [3]。

漏洞修复通常发生在不完备信息条件下。漏洞报告、检测器告警和概念验证程序（Proof of Concept, PoC）能够提供分析线索，但不一定直接指出应修改的程序位置，也不一定完整描述正确修复应满足的条件。在漏洞修复中，已知 PoC 不再触发，可能只是某个特定输入或执行分支被阻断，漏洞根因是否消除仍需独立验证。由此，AVR 实际上包含两个相互关联的问题：**如何根据有限证据推断应当修复什么，以及如何验证生成的补丁确实满足修复要求** [3, 12]。

LLM 为处理漏洞描述、源代码和修复经验提供了新的技术途径。进一步结合代码检索、文件编辑、编译执行与测试工具，模型能够在智能体框架中逐步完成定位、修改和反馈驱动的修复。PatchAgent 等工作已经将这些环节整合到交互式系统中，推动研究对象从给定代码片段上的补丁生成扩展至真实项目中的自动修复 [6]。交互能力的增强提出了更高的语义理解要求：系统仍需判断告警位置与根因位置之间的关系、识别跨函数依赖，并区分补丁通过当前验证与恢复完整安全要求之间的差别。

Rust 等系统编程语言使这一问题具有更丰富的语义层次。RustBelt 从形式化角度说明，基于不安全 Rust 代码（unsafe Rust code）实现的安全抽象需要满足相应验证条件，才能作为安全语言的可靠扩展 [28]；针对真实 Rust 程序的研究也揭示了内存与线程安全实践中的复杂问题 [29]。因此，Rust 漏洞修复不仅涉及局部条件检查，还可能涉及所有权、生命周期、类型与 trait 约束、异常清理和并发状态。基于上述背景，本节围绕传统修复方法、LLM 与智能体修复、补丁正确

性评价及 Rust 修复进展展开，重点讨论从漏洞表现恢复根因语义、从候选修改建立可信修复证据所面临的挑战。

2.2.2 研究现状与趋势

传统自动程序修复 (Automated Program Repair, APR) 已经形成了基于搜索、程序合成和语义约束等多种技术路线。GenProg 通过搜索程序变换空间寻找能够通过测试的候选补丁，SemFix 则利用符号执行和程序合成求解满足测试约束的修复表达式 [30, 31]。面向安全漏洞，ExtractFix 等工作进一步从漏洞执行中提取安全约束，并将其传播至候选修复位置以指导补丁生成 [32]。这类方法将修复从简单的测试通过推进到显式约束满足，但其能力通常受漏洞模型、程序分析精度和候选修复空间限制，对复杂跨过程语义和缺少明确错误信号的安全问题仍难以建立完整的修复条件。

大语言模型的发展显著扩展了漏洞修复中的候选补丁生成能力。VulRepair 等工作从历史漏洞补丁中学习漏洞代码到修复代码的映射；且零样本漏洞修复研究表明，通用代码模型无需针对特定漏洞类型训练，也能够根据漏洞代码和提示信息生成有效候选补丁 [33, 34]。此后研究进一步引入漏洞上下文和根因信息，例如 APPATCH 结合静态依赖分析、根因推理和历史修复经验组织模型所需的代码上下文 [11]。这些工作表明，LLM 修复正在由局部代码生成转向利用跨函数上下文和漏洞语义进行修复，但模型生成的补丁是否真正恢复安全性质仍需要程序级验证。

进一步地，修复系统开始从固定的生成流程发展为能够自主操作真实仓库的智能体。SWE-agent 展示了模型通过代码导航、文件编辑和程序执行完成复杂软件工程任务的能力，PatchAgent 等工作进一步将漏洞定位、补丁生成和测试反馈纳入持续的交互式修复过程 [6, 35]。这类方法使模型能够根据构建、漏洞复现和回归测试结果不断调整补丁，但多轮迭代也可能使补丁逐渐适配已有测试，而没有真正消除漏洞根因。因此，智能体自主性的提高并不能替代对补丁安全性的独立验证，修复位置、根因解释和最终安全性质需要分别考察。

与修复能力的发展相对应，真实漏洞基准逐渐从提供漏洞代码转向提供可复现的仓库级任务。Vul4J 为 Java 漏洞提供可执行的真实项目和漏洞证明测试，SEC-bench 则将类似任务扩展到真实 C/C++ 仓库，并探索自动化构建漏洞修复

案例 [36, 37]。这些基准为端到端修复提供了实验基础，但仅依赖已有 PoC 和功能测试仍面临经典的测试过拟合问题：补丁可以使已知触发失效并通过回归测试，却仍然没有恢复漏洞所要求的安全性质。

因此，近期研究开始加强补丁的独立安全验证。PVBench 利用官方修复新增的测试检查基础验证遗漏的漏洞相关行为，PatchBench 则通过额外漏洞输入和正常行为对照进一步检查补丁的安全性与功能保持 [38, 39]。这些工作表明，已知 PoC 消失并不足以证明漏洞被正确修复；同时，评估阶段能够使用的官方补丁和隐藏测试通常不应直接暴露给修复系统。因此，**修复过程能够获得的信息与独立评估所使用的安全证据需要明确隔离**，并进一步区分漏洞必要的安全要求与官方实现中的其他工程选择。

在这一方向上，本课题组前期构建了面向 Rust 真实漏洞的仓库级修复基准 **RRBench** [40]，包含 115 个可复现漏洞案例。RRBench 将补丁评价划分为公开验证和私有验证两个阶段：公开阶段提供漏洞触发 PoC 与功能测试，私有阶段利用修复系统不可见的安全性质和合法行为检查验证补丁是否真正恢复漏洞要求。对代表性修复智能体的实验表明，通过公开验证的补丁中仍有 23.4% 无法通过私有验证，并有 14 个案例未被任何配置正确修复。该结果说明，**端到端漏洞修复的关键问题已经不仅是生成能够通过测试的补丁，还包括如何理解漏洞根因、形成正确修复，并利用独立安全性质验证其正确性**，为后续研究“生成—验证”协同的漏洞修复方法提供了直接基础。

2.2.3 研究挑战

首先，复杂漏洞修复需要从不完备的定位信息中恢复**根因位置与必要上下文**。已有研究表明，漏洞触发位置与实际修复位置可能跨越不同函数，根因分析需要结合静态依赖和动态证据 [10, 11]。当上游仅提供崩溃位置、污点终点或局部告警时，系统仍需追踪状态的建立、传播和使用关系；同时，上下文过少可能遗漏关键依赖，过多又会引入无关信息。因此，关键问题是如何将初始定位线索转化为可验证的程序依赖证据，并据此动态确定分析和修复范围。

在此基础上，还需要保证**根因判断与修复策略之间的一致性**。漏洞类别和历史案例能够提供修复经验，但并不能直接确定目标程序中的缺陷对象、形成原因和正确干预位置。现有 LLM 修复方法已经利用根因推理和历史经验辅助补丁

生成 [11, 41], 但错误的早期解释也可能持续影响上下文选择和候选生成。因此, 根因假设需要接受控制流、数据流、接口约束和执行证据的检验, 并在证据冲突时重新定位或调整修复策略, 而不是围绕单一假设反复生成相似补丁。

进一步地, 正确修复要求恢复**跨接口和状态的安全性质**。真实漏洞的安全要求可能由多个函数、对象状态和调用阶段共同维护, 尤其在 Rust 等具有丰富类型和生命周期机制的系统语言中, 漏洞表现位置并不一定是安全约束应当建立的位置 [28, 29]。修复不仅需要消除已知触发, 还应确定安全性质由哪些操作共同维护以及是否存在绕过路径。对于部分漏洞, 增强类型或接口约束也可能构成有效修复, 因此评价时还需区分对危险行为的有意限制与对合法程序行为的意外破坏。

由此产生的核心问题是**如何将安全性质转化为独立、可执行的补丁验证**。已有研究通过额外漏洞输入、开发者新增测试和行为对照增强补丁验证 [38, 39], 但有限测试仍可能接受不完整修复, 过强的检查又可能拒绝不同于官方实现的正确方案。因此, 需要从漏洞根因中提炼必要的安全要求, 将其转化为可执行检查, 并同时设置合法行为对照。修复阶段可以利用公开测试和生成的反例持续迭代, 但最终验证应与修复过程保持信息隔离, 避免修复器针对隐藏判据过拟合。有限验证只能证明补丁满足已检查的安全要求, 而不能排除未覆盖执行空间中的残余风险。

最后, 智能体修复需要建立**统一且可复现的端到端评价标准**。现有研究在是否提供漏洞位置、是否要求仓库级分析以及采用测试、参考补丁还是人工审查等方面存在明显差异 [37, 39]。评价时应明确系统能够获得的信息, 分别记录补丁生成、构建、公开验证和独立安全验证结果, 并保留无补丁、环境失败和预算耗尽等情况。同时还需要控制历史补丁泄漏以及模型、工具链和执行环境差异, 并将上下文分析、候选生成和验证成本纳入评价, 从而区分模型能力、额外信息和分析机制各自带来的收益。

综上, 智能体漏洞修复已经从局部代码生成发展到真实仓库中的交互式分析, 但可信修复仍依赖两个相互衔接的核心能力: **一是从不完备信息中形成有证据支持的根因判断并生成相应修复, 二是将漏洞安全要求转化为独立、可执行且兼顾合法行为的验证条件**。本课题研究方向二围绕这一“生成-验证”协同过程展开, 研究根因分析、候选修复与安全性质验证之间的联系, 以提高复杂真实漏

洞自动修复的可靠性。

2.3 基于智能体的漏洞修复验证

2.3.1 研究背景

漏洞修复是软件安全保障的重要环节，其目标除消除已知漏洞的触发条件外，还需要在维持合法功能的前提下阻断危险行为。围绕这一目标，自动漏洞修复（Automated Vulnerability Repair, AVR）研究形成了漏洞分析、补丁生成与补丁验证三个主要环节，采用程序分析、约束求解、程序变换和学习等技术降低对人工修复的依赖 [3]。随着大语言模型的发展，修复过程进一步由单一补丁生成扩展为交互式任务执行，PatchAgent 将缺陷定位、补丁生成与验证组织到同一智能体中，利用语言服务器和验证工具支持多轮修复 [6]。这些研究主要回答“如何产生有效修复”，软件系统的实际安全状态还取决于修复能否被正确集成并传播至最终部署的程序。

从软件供应链的角度看，漏洞修复远超上游提交补丁这一局部开发活动，它贯穿上游项目、下游分支、发行版、设备厂商与最终用户，构成一个持续维护过程。Li 和 Paxson 对安全补丁的研究表明，补丁开发需要同时考虑修复完整性与功能回归，安全修复也可能仅涉及少量代码变化 [42]。针对修复在相关仓库之间的传播，Machiry 等提出 SPIDER，通过分析补丁的行为影响，识别适合向相关项目传播的修复 [43]；Zhang 等对 Android 内核补丁生态的研究进一步发现，修复在多层供应链中的传播可能存在较长延迟 [44]。因此，“补丁已经公开”“下游已经合入”与“部署产物实际包含修复”是三个需要分别确认的状态。

版本信息和更新公告能够辅助判断修复状态，其覆盖范围存在边界。一方面，下游维护者可能将安全修复回移植至较早版本，同时保留原有上游版本号，Red Hat 的安全回移植实践明确指出，仅依据上游版本号开展漏洞扫描可能产生误判 [45]。另一方面，闭源产品、固件和第三方二进制组件往往无法提供与部署产物完全对应的源码，使用者难以直接检查安全修复是否进入最终程序。由此，针对实际二进制开展独立的修复状态核查，成为连接补丁发布与安全维护的重要技术需求 [4]。

补丁存在性测试（Patch Presence Testing, PPT）正是针对这一需求提出的：给定已知漏洞的修复信息，以及补丁状态未知的目标程序，判断目标中是否包含

对应修复的实现证据。不同方法使用的参考材料有所区别，包括源码补丁、修复前后源码、参考二进制或中间表示等 [4, 46]。PPT 与通用二进制相似性检测的区别在于，后者主要寻找功能或结构相近的代码，前者则需要在高度相似的两个实现之间识别安全相关差异：一个边界条件、错误分支或参数处理的变化，可能不足以明显改变函数整体相似度，却足以改变其漏洞状态 [46]。

需要区分的是，**PPT 核查”特定修复是否体现在目标程序中”，补丁正确性验证考察”修复是否充分消除目标缺陷并维持合法行为”**。即使检测到参考修复已经存在，也不能据此证明参考补丁本身完整，更不能推导出目标程序不存在其他漏洞 [3, 4, 42]。因此，本节将 PPT 定位为漏洞修复生命周期中的部署侧验证环节，重点讨论其二进制证据提取、补丁语义匹配以及智能体化分析能力。

2.3.2 研究现状与趋势

早期补丁存在性测试（Patch Presence Testing, PPT）主要通过修复前后的局部语法和结构差异构造补丁特征，并在目标二进制中寻找对应证据。BinXray 和 PatchDiscovery 分别利用基本块映射、关键基本块及控制流结构描述补丁变化 [46, 47]。这类方法特征直观、匹配效率较高，但要求参考程序与目标程序之间保持较稳定的结构对应关系，容易受到编译优化、基本块重组和版本变化的影响。

为降低对具体指令形式的依赖，后续研究逐渐转向补丁语义建模。FIBER、PDiff 等方法利用符号执行、路径约束和程序状态描述补丁相关行为，PS³ 和 REACT 则进一步通过语义签名或中间表示增强不同编译配置下的补丁匹配能力 [4, 48–50]。这些方法提高了 PPT 对二进制表示变化的适应能力，但分析结果仍依赖符号执行、IR 恢复和特征抽象的准确性，并通常需要目标函数或候选范围等定位信息。因此，稳定的补丁语义表示与准确的目标定位仍是传统 PPT 的两个关键问题。

除编译差异外，补丁本身还会随软件版本持续演化。已有研究表明，后续版本可能通过重构检查逻辑、移动代码或改变数据流重新实现同一安全修复，从而削弱基于原始补丁特征的检测能力 [51]。这意味着 PPT 不仅需要适应同一实现经过不同编译过程产生的表示差异，还需要识别不同实现所保持的共同安全语义，检测对象也由具体补丁特征逐渐转向补丁所代表的安全行为。

大语言模型为跨源码与二进制表示的语义分析提供了新的途径。Lares 利用 LLM 将源码补丁切片映射到目标函数的反编译代码，并结合约束求解完成语义判断；PPTSP 则在语义规范化和关键路径提取后利用 LLM 处理难以通过传统特征区分的情况 [52, 53]。这些方法表明，模型能够辅助理解补丁语义并处理跨表示差异，但仍主要工作在预先给定的函数或候选范围内。从完整剥离二进制出发自主定位补丁相关代码、获取证据并形成最终判断，仍需要更完整的分析过程。

随着编码智能体的发展，PPT 可以进一步从固定分析流程转向交互式分析。智能体能够围绕补丁假设自主调用反汇编、检索和程序分析工具，在候选定位、局部检查和证据补充之间迭代。此时，系统能力不再仅由模型本身决定，工具接口、上下文信息、分析策略和资源预算都会影响最终结果 [35, 54]。因此，研究重点正在由单一补丁特征或模型输出扩展到完整的端到端分析过程，包括补丁定位、语义判断、证据获取以及分析成本。

围绕上述问题，本人前期已开展两项相互衔接的研究。首先，构建了包含 561 个 CVE、覆盖 10 个开源项目的 PPT 基准，并系统评估五款代表性工具，揭示了现有方法“高准确率、低决策成功率”的现象及其主要失败原因 [55]。进一步针对端到端场景，系统评估编码智能体在符号可见、剥离二进制、补丁演化和真实发行版软件包等条件下的 PPT 能力，发现前沿智能体仅使用基础二进制工具即可达到与专用方法相当的性能，但补丁定位和演化补丁识别仍是主要瓶颈 [56]。这些结果表明，PPT 的研究重点正在由**局部特征匹配**进一步转向**面向完整二进制的补丁定位、语义理解与证据验证**，为研究内容三提供了直接的研究基础。

2.3.3 研究挑战

首先，实际部署中的 PPT 需要解决**缺少定位提示时的补丁定位问题**。现有方法通常不同程度依赖符号、候选函数或外部定位结果 [4, 46, 52]，因此在已知函数范围内取得的匹配效果不能直接代表完整二进制上的端到端能力。分析系统需要从补丁语义出发定位相关函数和具体程序位置，并进一步确认相应检查是否真正作用于目标对象和危险操作；当存在多个相似候选时，还需要持续获取证据而非直接接受局部相似结果。

其次，**补丁演化会削弱原始修复特征与目标实现之间的对应关系**。已有语法

签名、符号执行和 IR 方法能够缓解编译带来的表示差异，但后续版本可能通过重构控制流、数据流或错误处理方式重新实现同一安全修复 [50, 51]。因此，需要区分“原始补丁模式消失”和“安全修复失效”，从具体代码特征进一步恢复补丁所表达的安全语义，并根据检查对象、错误传播和危险操作之间的关系判断演化后的实现是否仍满足修复要求。

进一步地，引入大模型后需要保证**模型推理与程序证据之间的一致性**。Lares、PPTSP 等工作表明模型能够辅助跨表示定位和语义判断 [52, 53]，但自然语言解释本身不能作为补丁存在性的充分证据。最终判断需要能够回溯到具体指令、控制流和数据关系，并区分补丁不存在、漏洞相关功能不适用以及证据不足等不同情况，避免将检索失败、工具异常或模型推测直接转化为漏洞状态。

同时，PPT 系统还需要建立**统一的端到端评价方法**。不同研究在数据集、符号信息、候选函数和编译条件等方面存在差异，单独比较 Accuracy 或 F1 难以反映实际能力。评价应分别记录定位、语义判断和完整流程中的错误、无法判定及工具失败，并明确系统能够获得的信息条件，从而判断失败究竟来自补丁语义本身、定位能力还是工程实现，而不是将所有失败归结为算法精度。

最后，智能体分析还需要权衡**分析能力与资源成本**。更强模型、更多分析工具和更长上下文能够扩大证据搜索空间，但并不必然提高最终检测效果 [35, 54]。因此，需要在统一任务条件下比较模型能力、漏洞信息、工具配置和资源预算，研究如何利用低成本线索完成初步定位，并仅在证据不足时调用更复杂的程序分析，同时评价检测正确性、完成率和资源消耗。

综上，面向实际部署的 PPT 仍需要解决**补丁定位、演化修复识别、证据约束下的语义判断以及端到端可靠评价**等问题。本课题研究内容三将围绕这些环节，进一步研究程序分析与智能体推理的协同机制，提高复杂二进制场景下补丁存在性检测的可靠性与实用性。

三 课题主要研究内容与预期目标

图 1 给出了本课题三个研究内容的总体概览。

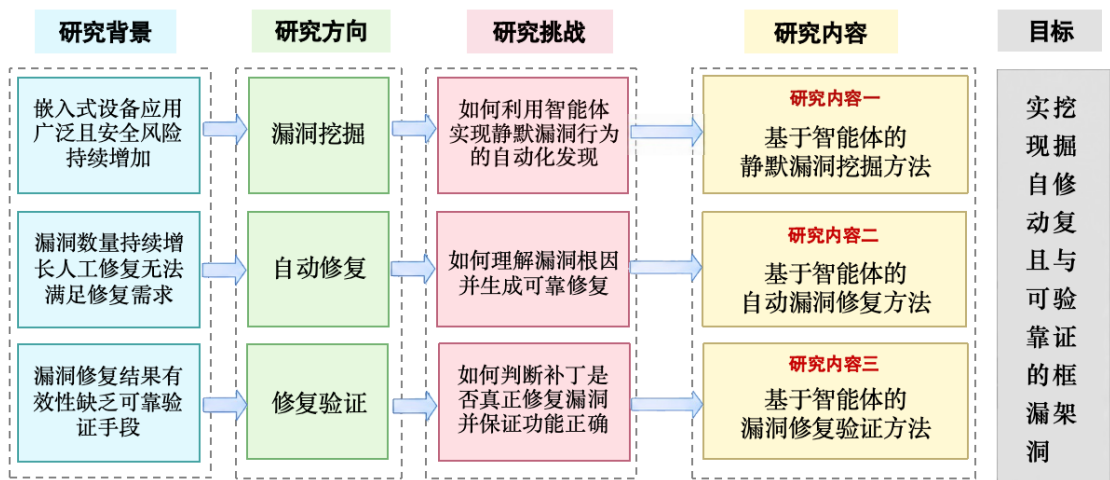


图 1 研究内容总览

3.1 研究目标

漏洞治理不仅需要发现潜在的安全缺陷，还需要针对漏洞形成有效修复，并确认修复满足相应安全要求、进入实际部署程序。仅发现漏洞或生成补丁，尚不足以确认目标系统中的相应风险已经消除。因此，本课题围绕“发现问题—生成修复—确认修复”的研究主线，探索大模型语义理解、智能体推理与程序分析相结合的方法，提升漏洞治理关键环节的自动化水平与结果可靠性。

在漏洞挖掘环节，本课题重点关注嵌入式设备与剥离符号二进制场景中的协议状态违例、行为逻辑错误和资源泄漏等问题。此类问题的发现不仅需要触达相关执行路径，还需要判断程序行为是否违反协议、接口或资源生命周期要求。为此，拟研究受限信息条件下的行为约束恢复与针对性测试方法，将复杂交互中的状态变化与安全要求联系起来，提高对不易通过崩溃信号直接识别的漏洞的发现能力。

在漏洞修复环节，漏洞报告所指向的异常位置未必是实际需要修改的根因位置，而上下文缺失与过早收敛于单一解释，也可能限制修复策略的选择。因此，本课题拟研究上下文增强与证据驱动的根本原因分析方法，结合历史修复经验开展多假设推理与多策略补丁生成，增强从漏洞表现追溯根因、从根因形成有效修改的能力，提高复杂漏洞自动修复的准确性与有效性。

在漏洞修复验证环节，补丁通过已知漏洞触发测试与功能测试，并不能充分说明漏洞根因已经消除 [40]；同时，参考补丁的存在也不能直接证明实际部署的二进制包含对应修复 [56]。为此，本课题拟从修复有效性与目标产物修复状态两个层面开展研究：一方面检查补丁是否满足漏洞相关安全要求并保持必要的合法行为，另一方面分析目标程序是否包含对应修复，提高对不完整修复、功能回归及修复缺失的识别能力。

基于上述目标，本课题形成**基于智能体的漏洞挖掘、基于智能体的漏洞自动修复、基于智能体的漏洞修复验证**三个研究内容。三者分别面向漏洞发现、候选补丁生成与修复结果确认，在任务目标上逐步衔接，共同为从漏洞识别到修复确认提供方法与技术支撑。

3.2 研究内容一：基于智能体的漏洞挖掘方法

主要研究内容：面向嵌入式设备与剥离符号二进制，协议状态违例、行为逻辑错误和资源泄漏等问题可能不立即产生崩溃，其触发与判定往往依赖多步交互、内部状态和资源生命周期要求，仅增加测试输入或代码覆盖难以充分发现此类漏洞。当前工作仍面临行为约束不完整、程序状态难以观察、违例路径难以触达以及错误判据不足等挑战。因此，本文拟提出行为约束引导与状态感知相结合的智能体漏洞挖掘方法。利用大模型分析协议文档、可获得的程序信息及设备交互结果，提取具有明确依据和适用条件的行为约束，建立约束与可观察事件之间的对应关系；结合状态转移和资源生命周期，指导智能体构造允许交互范围内的多步测试序列，并依据执行反馈调整探索方向；针对缺少直接错误信号的问题构造行为检查，通过独立回放与合法行为对照，确认候选缺陷及其安全影响。

预期目标：构建面向嵌入式设备与剥离符号二进制的智能体漏洞挖掘原型系统，实现行为约束提取、状态感知测试与候选漏洞复现，提高协议状态违例、行为逻辑错误和资源泄漏等问题的发现能力，形成包含触发条件、交互序列与安全影响的可复现漏洞案例。

3.3 研究内容二：基于智能体的漏洞自动修复方法

主要研究内容：漏洞报告提供的异常位置不一定是实际需要修改的根因位置，跨函数依赖与领域约定又使局部代码难以完整表达修复所需信息。当前工作

仍面临根因定位不准确、上下文缺失与噪声并存，以及模型过早接受单一解释、生成补丁同质化等挑战。因此，本文拟提出上下文增强与检索辅助的多假设智能体漏洞修复方法。围绕漏洞相关位置分析数据依赖与控制依赖，裁剪无关代码，补全关键定义、辅助函数和调用关系，重新定位实际修复区域；结合代码特征与根因描述检索历史修复案例，引导智能体提出并比较多种根因假设，减少对漏洞类别和固定修复模式的依赖；在根因分析基础上，从最小修改、安全实现替换与局部结构调整等角度生成候选补丁，提高对有效修复方案的覆盖能力。

预期目标：构建基于智能体的漏洞自动修复原型系统，实现从漏洞相关位置和项目源码出发的上下文构建、根因重定位与多策略补丁生成。在不预先提供准确修复位置的条件下，提高根因分析准确性与正确补丁生成能力，输出具有明确修改依据的候选修复方案。

3.4 研究内容三：基于智能体的漏洞修复验证方法

主要研究内容：漏洞修复验证既需要检查补丁是否满足漏洞相关安全要求并保持合法行为，也需要确认实际部署的目标程序是否包含对应修复。当前工作面临两方面挑战：已有触发测试与功能测试难以覆盖根因相关的必要安全条件，可能接受不完整修复；符号缺失、编译变换与版本演化则使修复在目标二进制中的定位和识别更加困难。因此，本文拟提出结合安全性质检查与二进制语义分析的智能体漏洞修复验证方法。一方面，从漏洞描述、相关代码与修复信息中提取必要安全条件，利用智能体构造独立于补丁生成过程的可执行检查与合法行为对照，并通过漏洞版本、参考修复及不完整修复校准判定能力，重点支持 Rust 中类型约束、资源管理和状态转换等不同性质的验证。另一方面，以修复意图引导目标二进制中的候选区域检索与程序位置定位，结合控制关系、数据依赖和调用上下文，识别不同实现形式下的对应修复，分别给出修复有效性与目标修复状态的结论。

预期目标：构建基于智能体的漏洞修复验证原型系统，实现安全性质检查的自动构造与校准，以及目标二进制修复状态的端到端核验。提高对不完整修复、合法行为受损和目标修复缺失的识别能力，并为验证结论提供可复现的反例或具体代码依据，明确证据不足时的判定边界。

四 拟采用的研究方法、技术路线、实验方案及其可行性分析

4.1 基于智能体的漏洞挖掘方法

4.1.1 问题定义与总体技术框架

本研究面向嵌入式设备与剥离符号二进制，重点挖掘协议状态违例、跨调用行为逻辑错误和资源泄漏等不易通过崩溃信号识别的问题。针对行为约束不完整、内部状态难以观察以及错误判据不足等挑战，本文拟提出行为约束引导的状态探索与主动观测方法，将“如何触发潜在错误”与“如何观察并判断错误”联合考虑，通过针对性交互获得可复现的行为违例。

方法以目标设备或可执行二进制，以及能够获得的协议文档、接口说明、配置和正常交互样例为输入，不要求预先提供漏洞位置、触发程序或官方补丁。对于二进制可分析且具备运行时观察条件的目标，结合静态分析与动态观察辨识安全相关状态；对于仅能进行设备交互的目标，则利用响应、后续操作结果及可用诊断信息推断状态，不假设能够获取内部变量或代码覆盖。按照“行为约束构建—状态辨识与主动观测—交互生成与反馈优化—候选漏洞确认”的路线开展，技术路线如图 2 所示，输出包含约束依据、触发条件、交互序列及安全影响的漏洞报告。



图 2 基于智能体的漏洞挖掘技术路线

4.1.2 行为约束引导的状态探索与主动观测

首先，构建具有明确依据和适用范围的局部行为约束。利用大模型分析协议规范、接口说明及历史缺陷模式，提取约束对象、适用前提、触发事件和预期结果，再结合目标实现与交互信息检查其适用条件及事件映射。例如，资源约束不

仅描述申请与释放操作，还需明确资源所属的会话、生命周期结束条件，以及允许延迟回收的例外。规则构建区分规范要求与实现观察，不将目标当前表现直接视为正确行为；缺少充分依据的规则保留为待验证假设。

其次，建立行为约束与目标可观察状态之间的对应关系。对于二进制目标，围绕约束涉及的操作定位状态建立、更新、使用和清理位置，并在具备条件时利用运行轨迹核实对象关系。对于黑盒设备，当即时响应不足以辨识状态时，由智能体构造具有区分能力的后续交互。例如，在规范要求注销后权限失效的场景中，除检查注销响应外，还需通过后续受保护操作观察原会话权限是否残留。系统根据结果更新候选状态假设；对可能改变被测状态的探测，通过相同前缀的独立回放进行对照，减少观察操作本身的干扰。

在此基础上，联合构造状态建立、条件扰动与后续观察序列，将一次测试表示为

$$T = P \parallel \Delta \parallel Q, \quad (1)$$

其中， P 为建立目标状态的交互前缀， Δ 为针对特定约束的扰动， Q 为用于辨识结果的观察后缀， $\parallel$ 表示序列连接。测试在保持无关语法和依赖条件的同时，改变消息顺序、取消时机或断连行为等目标条件，避免因前置步骤无效而始终停留在外围检查。对于资源累积问题，在同一运行周期中重复相关交互，再检查清理后的资源占用与后续申请能力。所有触发均限定在预设的交互能力和有效环境条件内，不以任意修改内部状态代替真实可达过程。

执行过程中，智能体负责约束解释、交互规划和观察策略调整，测试引擎负责批量变异、执行与结果采集。反馈除可获得的代码覆盖外，还包括约束前提是否成立、目标事件是否发生、状态假设是否得到区分以及资源回收条件是否满足。当前提未建立时补充交互前缀，扰动未触及目标行为时调整测试条件，结果无法判定时增加观察后缀，从而将测试预算用于具体的触发或观察缺口。

最后，根据约束类型构造相应判据：协议类检查状态转移及异常消息处理是否符合要求，跨调用逻辑类检查取消、回滚和对象隔离等行为关系，资源类检查生命周期结束后的资源残留与容量恢复。判定针对目标的实际行为，而不是将测试输入不合法直接视为漏洞。资源增长或等待超时仅作为候选信号，需要结合明确的回收条件与正常路径对照排除合法缓存和延迟清理。对候选问题进行独立回放、序列最小化和安全影响分析，排查规约误解与环境偏差；观察不足时保留

不确定结论，不以模型判断替代可验证证据。

4.1.3 实验方案与可行性分析

实验拟按照可控软件对象、剥离符号二进制和真实设备逐步开展。在可控对象上利用源码和完整执行信息建立评价依据，但按实验设置限制被测系统能够访问的信息；随后分别评价剥离二进制与设备黑盒条件下的发现能力。测试对象覆盖协议状态、跨调用行为及资源生命周期相关缺陷，已知漏洞回溯与未知漏洞发现分别报告，目标漏洞的官方补丁和事后测试不进入发现阶段。

对比方法包括适用的状态感知模糊测试、规则驱动测试，以及使用相同模型、工具权限和预算的通用智能体。通过移除行为约束、主动观测和反馈调度等模块，评价各项设计的贡献，并比较持续多轮测试与逐轮重置对资源问题发现的影响。主要指标包括独立确认的缺陷与安全漏洞数量、首次确认时间、误报比例、无法判定比例及完整分析成本；状态覆盖与有效交互比例用于解释方法效果，不替代最终漏洞判定。

实现上拟复用二进制分析、模糊测试和设备交互工具，将新增研究集中于行为约束映射、触发与观察序列的联合生成，以及分类判据的可靠执行。初期选择具备明确协议或接口说明、可重复运行的目标验证核心机制，再逐步扩展至信息和观察能力受限的设备。对于环境难以复现或约束无法确认的案例，记录相应限制并分开分析，以控制实施风险并明确方法适用范围。

4.2 基于智能体的漏洞自动修复方法

4.2.1 问题定义与总体技术框架

本研究面向已报告漏洞的源码自动修复，重点解决漏洞表现位置与实际修复位置不一致、上下文信息不完整，以及模型根因推理过早收敛和补丁生成同质化等问题。本文拟采用分层上下文构建、检索增强的多假设根因分析与多策略补丁生成相结合的方法，逐步完成从漏洞相关位置到根因解释、再到候选修复的分析过程，减少对准确修复位置和固定补丁模式的依赖。

方法以项目源码、漏洞相关位置及对应的漏洞类别为输入。其中，漏洞相关位置可以是异常触发或表现位置，不要求预先指出应修改的语句或函数。整体按照“漏洞上下文构建—多假设根因分析—多策略补丁生成”的路线开展，通过不

同分析角色完成源码裁剪、依赖补全、根因推理和候选生成，无需针对修复任务进行模型微调。系统输出细化后的修复区域、根因分析报告及候选补丁集合，补丁是否满足安全要求则由后续独立验证确认。整体技术路线如图 3 所示。

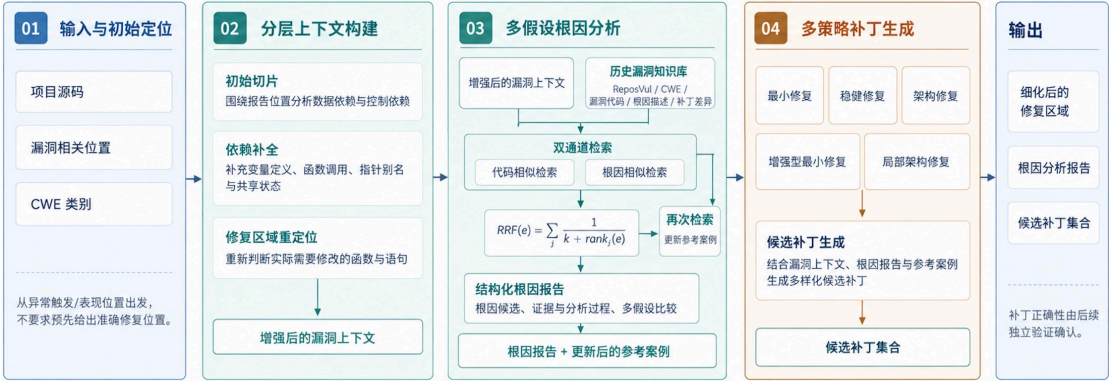


图 3 基于智能体的漏洞自动修复技术路线

4.2.2 上下文增强与多假设推理驱动的补丁生成

首先，采用分阶段分析构建漏洞相关上下文。以报告位置为起点，引导大模型沿前向和后向分析数据依赖与控制依赖，保留与漏洞传播、危险操作及其前置条件相关的代码，裁剪无关实现，形成初始切片。随后，由依赖分析角色检查缺失的变量定义、函数调用及隐式状态关联，结合指针别名、共享状态和操作副作用，从源码仓库中补充相关代码。最后，由安全分析角色检查现有上下文是否足以解释漏洞，对缺失的辅助函数、数据结构和领域逻辑继续检索，并重新判断实际需要修改的程序区域。该过程以降低噪声、覆盖根因和保留必要的定义-使用关系为目标，而不是将报告位置直接作为补丁插入位置。

其次，构建支持根因分析的历史漏洞知识库。以 ReposVul [57] 中的真实漏洞修复案例为基础，保存漏洞类别、漏洞代码、根因描述和补丁差异，分别对代码与根因文本进行向量化，并按漏洞类别组织检索分区。针对当前漏洞，智能体先依据增强后的上下文生成初始根因解释，但仅将其作为检索线索，不直接作为最终结论。随后，在对应类别中分别使用漏洞代码和初始根因描述开展余弦相似度检索，得到代码相似性与根因相似性两个排序列表。

为结合不同检索信号，采用倒数排名融合（Reciprocal Rank Fusion，RRF）

[58] 计算案例的综合相关性：

$$\text{RRF}(e) = \sum_{j \in \{\text{code}, \text{rca}\}} \frac{1}{k + \text{rank}_j(e)}, \quad (2)$$

其中， e 表示历史案例， $\text{rank}_j(e)$ 表示其在代码或根因检索通道中的排名， k 为平滑常数。该方法在排序层面融合信息，避免直接拼接较长代码与较短根因描述造成信息不均衡。选取融合排名靠前的案例，将其根因说明和补丁差异作为参考，引导智能体提出多种候选根因解释，再结合初始分析形成结构化根因报告。漏洞类别用于限定检索范围，而不直接决定当前漏洞的根因或修复模板。

在根因报告形成后，利用更新后的解释再次检索修复案例。具体而言，重新编码最终根因报告，更新根因通道的案例排序，并与已有代码通道排序进行融合，获得更贴近当前漏洞机制的修复参考。与仅在分析开始时检索一次相比，这一步使补丁生成所使用的历史经验能够随根因理解的变化而调整，减少初始解释偏差对后续生成的持续影响。

最后，通过五种互补策略引导补丁生成。最小修复以低侵入性修改直接处理核心根因；稳健修复采用成熟的安全实现替换危险编码模式；架构修复通过设计层面的调整减少产生危险行为的可能；增强型最小修复在局部修改基础上补充稳健性检查；局部架构修复结合安全实现替换与局部设计改进，在修改范围和结构调整之间取得平衡。智能体结合漏洞上下文、根因报告与更新后的参考案例，分别生成不同策略下的候选补丁，使候选差异体现在实际修复方式，而不只是代码表述变化。

各候选补丁保留对应的根因解释和修改依据，形成供后续验证的修复集合。本阶段关注有效修复方案的生成与覆盖，不以模型对补丁的自我评价代替正确性验证，也不预设修改最少或最接近官方补丁的方案必然正确。

4.2.3 实验方案与可行性分析

实验拟采用 APPATCH [11] 中的真实漏洞案例评价整体修复能力，并利用 InterPVD [10] 中触发位置与修复位置跨函数分离的案例，评价不准确定位条件下的上下文构建与根因重定位能力。历史知识库与测试集按漏洞实例去重，排除测试漏洞对应的修复案例，避免检索直接暴露目标补丁。系统可访问漏洞源码、报告位置和类别信息，参考补丁仅用于评价，不进入目标漏洞的生成过程。

对比方法包括直接提示生成、APPATCH [11], 以及 VulRepair [33]、VulMaster [59] 等修复方法。其中, 提示式方法在相同基础模型和候选生成预算下进行比较, 训练式方法则按其模型配置独立报告。主要指标包括修复召回率、补丁准确率和 F1 值: 修复召回率衡量至少获得一个正确候选的漏洞比例, 补丁准确率衡量生成候选中正确补丁的比例; 同时统计根因分析准确率、不同模型下的表现及单例时间和推理成本。

4.3 基于智能体的漏洞修复验证方法

4.3.1 问题定义与总体技术框架

漏洞自动修复技术的发展使得大语言模型能够根据漏洞描述、源码和测试反馈生成候选补丁, 但如何判断生成补丁是否真正消除了漏洞根因, 仍然是自动修复走向实际应用面临的关键问题。已有方法通常依赖漏洞 PoC、功能测试或与官方补丁的代码相似性进行评价, 但这些信息只能反映特定触发路径或实现形式, 无法充分说明补丁是否恢复了漏洞对应的安全性质。因此, 本文关注漏洞修复验证问题, 即在未知参考补丁实现细节的情况下, 判断候选补丁是否真正满足漏洞修复要求。

具体而言, 漏洞修复验证包含两个层面的目标: 一方面, 需要判断候选补丁是否消除了漏洞根因, 并保持程序原有合法行为; 另一方面, 需要判断实际部署程序或二进制组件是否已经包含对应的安全修复。前者关注修复有效性, 后者关注修复状态核验, 二者均需要建立漏洞语义、补丁语义与程序行为之间的对应关系。

针对上述问题, 本文拟提出基于智能体的漏洞修复验证方法, 通过构建公平、可复现的修复验证基准, 并设计面向补丁语义理解和证据驱动分析的智能体框架, 实现自动化修复结果评估。整体技术路线包括: 首先构建覆盖真实漏洞、修复版本和安全性质验证信息的评测基准; 其次利用智能体从漏洞描述和补丁信息中理解修复意图, 并结合程序分析工具定位和验证目标程序中的安全修复证据; 最后通过独立验证机制输出修复状态及对应分析依据。整体技术路线如图 4 所示。



图 4 基于智能体的漏洞修复验证技术路线

4.3.2 评测基准构建与漏洞修复验证框架

现有漏洞修复评测通常依赖公开 PoC 和功能测试判断补丁有效性，但公开测试只能覆盖漏洞的一种具体表现，容易导致补丁针对测试输入过拟合，而未真正恢复漏洞相关安全约束。同时，不同研究采用的数据来源、漏洞类型、输入信息和评价标准存在较大差异，难以公平比较不同修复方法的实际能力。因此，本文拟构建面向智能体漏洞修复验证的公平评测基准。

基准构建首先从真实开源项目中收集具有完整漏洞证据链的案例，包括漏洞版本、修复提交、漏洞描述、触发样例以及运行环境。随后，以漏洞修复提交为依据构造漏洞前后版本，并通过源码级修改检查和二进制级构建验证保证样本标签可靠。针对不同软件版本、编译配置和部署环境，构建多样化测试场景，模拟真实软件供应链中的修复状态核验需求。

为了避免仅通过 PoC 判断修复效果，本文进一步引入基于安全性质的独立验证机制。针对每个漏洞案例，从漏洞根因、修复意图以及相关代码行为中提取需要恢复的安全条件，并将其转化为可执行验证规则。验证规则与修复系统的信息边界严格隔离，修复智能体只能获得漏洞描述、源码和公开测试反馈，而无法访问隐藏验证条件和参考补丁。最终通过公开测试和独立安全验证共同判断补丁是否正确，避免将“阻断已知触发”误认为“完成漏洞修复”。

在此基础上，本文设计基于智能体的漏洞修复验证框架。该框架以漏洞描

述、补丁信息以及目标程序为输入，通过 Agent Harness 组织模型推理、工具调用和证据管理流程。首先，智能体分析漏洞补丁差异，提取漏洞根因、修复意图以及可观测语义特征；随后调用二进制分析和程序分析工具，在目标程序中定位与补丁相关的代码区域，并检查目标程序是否体现对应安全行为；最后，智能体基于工具输出生成结构化验证结果，包括修复状态、相关代码位置以及支持该结论的分析证据。

针对补丁实现可能随版本演化的问题，验证过程不依赖固定代码模式匹配，而关注补丁所实现的安全语义。例如，同一漏洞修复可能由新增检查、错误传播方式调整或代码结构重构实现，系统需要判断不同实现是否达到相同安全目标。同时，对于缺少充分证据的情况，框架允许输出无法判定结果，避免基于不完整信息产生错误结论。

4.3.3 实验方案与可行性分析

实验拟在构建的漏洞修复验证基准上，将本文提出的智能体验证框架与通用代码智能体进行比较，在相同模型能力、输入信息、工具权限和分析预算下，评估不同 Agent Harness 设计对漏洞修复验证能力的影响。实验覆盖符号可用、符号剥离、编译配置变化以及修复演化等真实场景，综合准确率、决策成功率、无法判定比例以及分析成本评价方法效果。

同时，通过消融实验分析补丁语义理解、目标定位、证据管理和验证反馈等模块的作用，研究不同组件对于修复状态判断和错误分析能力的贡献。实验过程中保留无法判定和运行失败情况，避免仅统计成功输出案例导致评价偏差，从而更加全面地刻画基于智能体漏洞修复验证方法的能力边界。

五 已有科研基础与所需的科研条件

5.1 已有科研基础

本人围绕软件安全、漏洞分析与智能体安全方向开展了较为系统的文献调研和研究积累，已阅读并跟踪漏洞挖掘、自动漏洞修复、补丁验证、大语言模型与智能体应用于软件安全等领域的大量国内外研究工作，对传统程序分析方法、基于大模型的智能分析方法以及智能体驱动的软件安全研究范式具有较好的知识基础。通过前期调研，掌握了漏洞治理过程中漏洞发现、自动修复和修复验证等关键环节的发展现状与主要技术挑战，为后续开展基于智能体的漏洞挖掘与修复研究奠定了理论基础。

在漏洞修复验证方向，已开展了较为深入的研究工作。针对补丁存在性测试领域长期缺乏系统性评估、现有方法在真实部署场景下能力边界不明确的问题，完成了《A Comprehensive Empirical Analysis of Patch Presence Testing: Capabilities, Limitations, and Paths Forward》研究工作，并已被 *ISSTA 2026* 录取 [55]。该工作首次针对补丁存在性测试开展系统性实证分析，构建大规模、高质量评测数据集，对现有代表性方法进行统一评估，分析影响检测效果的关键因素及失败原因，为后续智能体方法在补丁验证领域的发展提供了重要参考。

在上述研究基础上，进一步探索了基于智能体的端到端补丁存在性测试方法，完成了《Understanding Coding Agents for Patch Presence Testing》研究工作，目前已投稿。该工作将研究对象从传统专用分析工具扩展到通用编码智能体，系统分析智能体在补丁存在性测试任务中的能力边界，并提出面向该任务的智能体框架设计。实验结果表明，基于智能体的方法能够在真实二进制分析场景中达到与现有专用工具相当的性能，同时揭示了补丁定位和补丁演化识别等关键挑战，为后续研究智能体驱动的软件安全分析提供了实践基础。

在漏洞自动修复方向，已参与构建面向真实 Rust 漏洞的仓库级智能体修复评测基准《RRBench: A Repository-Level Benchmark for Agentic Repair of Real-World Rust Vulnerabilities》，目前已投稿。该工作针对现有漏洞自动修复评测中仅依赖 PoC 和功能测试、难以判断补丁是否真正恢复漏洞安全性质的问题，构建了包含真实漏洞案例、公开修复任务和独立安全验证机制的评测基准。通过对现有通用智能体修复能力的系统评估，发现当前智能体虽然能够生成大量表面

有效的补丁，但仍存在根因理解不足、安全性质恢复不完整等问题。该工作为进一步研究基于智能体的漏洞自动修复方法、提升补丁生成可靠性提供了数据基础和问题依据。

综上，本人已在漏洞修复验证和智能体驱动漏洞分析方向形成了一定研究积累。其中，补丁存在性测试相关工作建立了对修复状态验证问题的系统认识，智能体补丁验证工作探索了端到端智能体解决方案，而漏洞修复评测基准工作进一步揭示了当前自动修复方法面临的关键挑战。这些研究基础为本课题后续围绕漏洞挖掘、漏洞自动修复以及漏洞修复验证三个方向开展深入研究提供了支撑。

5.2 所需科研条件

本课题研究需要的软件环境主要包括漏洞分析工具、程序分析框架、二进制分析工具以及大语言模型智能体开发环境。在漏洞挖掘方向，需要使用静态分析、动态分析、模糊测试和二进制分析工具，支持嵌入式设备、固件及复杂软件系统中的漏洞发现研究；在漏洞自动修复和修复验证方向，需要构建基于大语言模型的智能体框架，并结合代码分析、编译测试和漏洞验证环境开展实验。

硬件方面，需要具备支持大规模程序分析、大语言模型调用以及智能体实验运行的计算资源，包括高性能 CPU、GPU 服务器以及存储环境。同时，需要收集和维护真实开源项目、漏洞数据库、固件样本以及软件供应链组件等实验数据，为方法验证和系统评估提供基础。

目前，课题组已具备开展相关研究所需的软件安全研究环境，包括漏洞分析工具链、服务器计算资源以及大规模软件项目分析环境。同时，本人前期已积累漏洞数据构建、二进制分析、智能体框架开发和实验评测经验，能够支撑本课题后续研究工作的开展。

六 研究工作计划与进度安排

结合课题研究目标、技术路线及已有科研基础，本课题围绕漏洞修复验证、漏洞自动修复和漏洞挖掘三个方向开展研究：

1. **2025.09 – 2026.07：开展漏洞修复验证研究。**完成补丁存在性测试系统评测与基于智能体的端到端补丁验证研究，形成相关论文成果。
2. **2026.08 – 2027.03：开展漏洞自动修复研究。**面向真实漏洞场景，研究智能体漏洞修复中的根因分析、补丁生成与修复验证方法，完善漏洞修复评测基准。
3. **2027.04 – 2027.12：开展智能体漏洞挖掘研究。**面向嵌入式设备、固件和受限二进制环境，研究基于行为约束与状态分析的漏洞自动发现方法。
4. **2028.01 – 2028.06：完成论文总结与博士答辩。**整理研究成果，完成博士学位论文撰写、修改及答辩工作。

参考文献

- [1] Center N C S R. Vulnerability — glossary [EB/OL]. 2026. <https://csrc.nist.gov/glossary/term/vulnerability>.
- [2] Householder A D, Coughlin E, Kennedy-McHugh K, et al. The CERT guide to coordinated vulnerability disclosure: CMU/SEI-2017-SR-015 [R]. CERT Division, Software Engineering Institute, Carnegie Mellon University, 2017.
- [3] Hu Y, Li Z, Shu K, et al. Sok: Automated vulnerability repair: Methods, tools, and assessments [C]//34th USENIX Security Symposium. 2025: 4421-4440.
- [4] Zhang H, Qian Z. Precise and accurate patch presence test for binaries [C]//27th USENIX Security Symposium. 2018: 887-902.
- [5] Google Project Zero and DeepMind. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code [EB/OL]. 2024. <https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html>.
- [6] Yu Z, Guo Z, Wu Y, et al. PatchAgent: A practical program repair agent mimicking human expertise [C]//34th USENIX Security Symposium. 2025: 4381-4400.
- [7] Liu Y, Liu Z, Yu Z, et al. Towards generality: Task-adaptive binary analysis via semantic retrieval and verifiable reasoning [C]//35th USENIX Security Symposium. 2026.
- [8] Scharnowski T, Bars N, Schloegel M, et al. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing [C]//31st USENIX Security Symposium. 2022: 1239-1256.
- [9] Barr E T, Harman M, McMinn P, et al. The oracle problem in software testing: A survey [J/OL]. IEEE Transactions on Software Engineering, 2015, 41(5): 507-525. DOI: [10.1109/TSE.2014.2370955](https://doi.org/10.1109/TSE.2014.2370955).
- [10] Li Z, Wang N, Zou D, et al. On the effectiveness of function-level vulnerability detectors for inter-procedural vulnerabilities [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639218](https://doi.org/10.1145/3597503.3639218).
- [11] Nong Y, Yang H, Cheng L, et al. APPATCH: Automated adaptive prompting large language models for real-world software vulnerability patching [C]//34th USENIX Security Symposium. 2025: 4481-4500.
- [12] Smith E K, Barr E T, Le Goues C, et al. Is the cure worse than the disease? overfitting in automated program repair [C/OL]//Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015: 532-543. DOI: [10.1145/2786805.2786825](https://doi.org/10.1145/2786805.2786825).
- [13] Manès V J M, Han H, Han C, et al. The art, science, and engineering of fuzzing: A survey [J/OL]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331. DOI: [10.1109/TSE.2019.2946563](https://doi.org/10.1109/TSE.2019.2946563).

- [14] Serebryany K, Bruening D, Potapenko A, et al. AddressSanitizer: A fast address sanity checker [C]//2012 USENIX Annual Technical Conference. 2012: 309-318.
- [15] Mozilla. Frontier-model-assisted vulnerability discovery in Firefox [Z]. 2026.
- [16] Shoshitaishvili Y, Wang R, Hauser C, et al. FIRMALICE: Automatic detection of authentication bypass vulnerabilities in binary firmware [C]//Network and Distributed System Security Symposium (NDSS). 2015.
- [17] Pham V T, Böhme M, Roychoudhury A. AFLNet: A greybox fuzzer for network protocols [C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 460-465.
- [18] Natella R. StateAFL: Greybox fuzzing for stateful network servers [C]//2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2022.
- [19] Meng R, Mirchev M, Böhme M, et al. Linear-time temporal logic guided greybox fuzzing [C/OL]//Proceedings of the 44th International Conference on Software Engineering. 2022: 765-777. DOI: [10.1145/3510003.3510082](https://doi.org/10.1145/3510003.3510082).
- [20] Xia C S, Palma F, Jia M L, et al. Fuzz4All: Universal fuzzing with large language models [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024. DOI: [10.1145/3597503.3639121](https://doi.org/10.1145/3597503.3639121).
- [21] Meng R, Mirchev M, Böhme M, et al. Large language model guided protocol fuzzing [C]//Network and Distributed System Security Symposium (NDSS). 2024.
- [22] Ji J, Zhang C, Gan S, et al. FirmAgent: Leveraging fuzzing to assist LLM agents with IoT firmware vulnerability discovery [C/OL]//Network and Distributed System Security Symposium (NDSS). 2026. DOI: [10.14722/ndss.2026.231943](https://doi.org/10.14722/ndss.2026.231943).
- [23] Yang T, Qin Y, Zhang L, et al. BSFuzzer: Context-aware semantic fuzzing for BLE logic flaw detection [C]//Network and Distributed System Security Symposium (NDSS). 2026.
- [24] Song X, Pei L, Wu J, et al. ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification [C]//Network and Distributed System Security Symposium (NDSS). 2026.
- [25] Wang R, Li T, Li J. TyPatch: Transforming patches into tpestate rules for kernel bug detection [Z]. 2026.
- [26] Yang R, He Y, Xu Y, et al. LLM-based invariant testing for software functional bugs [C]//2026 IEEE International Symposium on Software Reliability Engineering (ISSRE). 2026.
- [27] Wang C, Liu J, Peng X, et al. Boosting static resource leak detection via LLM-based resource-oriented intention inference [C]//Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. 2025.
- [28] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: Securing the foundations of the Rust pro-

- programming language [J/OL]. Proceedings of the ACM on Programming Languages, 2018, 2 (POPL): 1-34. DOI: [10.1145/3158154](https://doi.org/10.1145/3158154).
- [29] Qin B, Chen Y, Yu Z, et al. Understanding memory and thread safety practices and issues in real-world Rust programs [C/OL]//Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. 2020: 763-779. DOI: [10.1145/3385412.3386036](https://doi.org/10.1145/3385412.3386036).
- [30] Le Goues C, Nguyen T, Forrest S, et al. GenProg: A generic method for automatic software repair [J]. IEEE Transactions on Software Engineering, 2012, 38(1): 54-72.
- [31] Nguyen H D T, Qi D, Roychoudhury A, et al. SemFix: Program repair via semantic analysis [C/OL]//2013 35th International Conference on Software Engineering. 2013: 772-781. DOI: [10.1109/ICSE.2013.6606623](https://doi.org/10.1109/ICSE.2013.6606623).
- [32] Gao X, Wang B, Duck G J, et al. Beyond tests: Program vulnerability repair via crash constraint extraction [J/OL]. ACM Transactions on Software Engineering and Methodology, 2021, 30 (2): 1-27. DOI: [10.1145/3418461](https://doi.org/10.1145/3418461).
- [33] Fu M, Tantithamthavorn C, Le T, et al. VulRepair: A T5-based automated software vulnerability repair [C/OL]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 935-947. DOI: [10.1145/3540250.3549098](https://doi.org/10.1145/3540250.3549098).
- [34] Pearce H, Tan B, Ahmad B, et al. Examining zero-shot vulnerability repair with large language models [C/OL]//2023 IEEE Symposium on Security and Privacy. 2023: 2339-2356. DOI: [10.1109/SP46215.2023.10179420](https://doi.org/10.1109/SP46215.2023.10179420).
- [35] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C]//Advances in Neural Information Processing Systems: volume 37. 2024: 50528-50652.
- [36] Bui Q C, Scandariato R, Díaz Ferreyra N E. Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques [C/OL]//Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 464-468. DOI: [10.1145/3524842.3528482](https://doi.org/10.1145/3524842.3528482).
- [37] Lee H, Zhang Z, Lu H, et al. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks [C]//Advances in Neural Information Processing Systems: volume 38. 2025.
- [38] Yu Z, Shi W, Sun X, et al. Patch validation in automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2603.06858](https://doi.org/10.48550/arXiv.2603.06858).
- [39] Shen C, Li J, Mahajan A, et al. PatchBench: Evaluating AI agents for vulnerability patching [EB/OL]. 2026. DOI: [10.48550/arXiv.2609.04075](https://doi.org/10.48550/arXiv.2609.04075).

- [40] Anonymous. RRBench: A repository-level benchmark for agentic repair of real-world Rust vulnerabilities [Z]. 2026.
- [41] Wang H, Basque Z L, Hu J, et al. Root-cause-driven automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2605.04251](https://doi.org/10.48550/arXiv.2605.04251).
- [42] Li F, Paxson V. A large-scale empirical study of security patches [C/OL]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 2017: 2201-2215. DOI: [10.1145/3133956.3134072](https://doi.org/10.1145/3133956.3134072).
- [43] Machiry A, Redini N, Camellini E, et al. SPIDER: Enabling fast patch propagation in related software repositories [C/OL]//2020 IEEE Symposium on Security and Privacy. 2020: 1562-1579. DOI: [10.1109/SP40000.2020.00038](https://doi.org/10.1109/SP40000.2020.00038).
- [44] Zhang Z, Zhang H, Qian Z, et al. An investigation of the Android kernel patch ecosystem [C]//30th USENIX Security Symposium. 2021: 3649-3666.
- [45] Red Hat. Security backporting practice [EB/OL]. 2026. <https://access.redhat.com/security/updates/backporting>.
- [46] Xu Y, Xu Z, Chen B, et al. Patch based vulnerability matching for binary programs [C/OL]//Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2020: 376-387. DOI: [10.1145/3395363.3397361](https://doi.org/10.1145/3395363.3397361).
- [47] Xu X, Zheng Q, Yan Z, et al. PatchDiscovery: Patch presence test for identifying binary vulnerabilities based on key basic blocks [J/OL]. IEEE Transactions on Software Engineering, 2023, 49(12): 5279-5294. DOI: [10.1109/TSE.2023.3332732](https://doi.org/10.1109/TSE.2023.3332732).
- [48] Jiang Z, Zhang Y, Xu J, et al. PDiff: Semantic-based patch presence testing for downstream kernels [C/OL]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. 2020: 1149-1163. DOI: [10.1145/3372297.3417240](https://doi.org/10.1145/3372297.3417240).
- [49] Zhan Q, Hu X, Li Z, et al. PS³: Precise patch presence test based on semantic symbolic signature [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639134](https://doi.org/10.1145/3597503.3639134).
- [50] Zhan Q, Hu X, Xia X, et al. REACT: IR-level patch presence test for binary [C/OL]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 381-392. DOI: [10.1145/3691620.3695012](https://doi.org/10.1145/3691620.3695012).
- [51] Xie Z, Wen M, Wei Z, et al. Unveiling the characteristics and impact of security patch evolution [C/OL]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 1094-1106. DOI: [10.1145/3691620.3695488](https://doi.org/10.1145/3691620.3695488).
- [52] Li S, Zheng Y, Li H, et al. Lares: LLM-driven code slice semantic search for patch presence testing [C/OL]//2025 40th IEEE/ACM International Conference on Automated Software Engineering. 2025: 919-931. DOI: [10.1109/ASE63991.2025.00081](https://doi.org/10.1109/ASE63991.2025.00081).

- [53] Xu C, Luo S, Duan X, et al. PPTSP: Patch presence test via semantic normalization and key path extraction [J/OL]. *Software Quality Journal*, 2026, 34(2). DOI: [10.1007/s11219-026-09760-9](https://doi.org/10.1007/s11219-026-09760-9).
- [54] Kapoor S, Stroebl B, Siegel Z S, et al. AI agents that matter [EB/OL]. 2024. DOI: [10.48550/arXiv.2407.01502](https://doi.org/10.48550/arXiv.2407.01502).
- [55] Zhang X, Zheng Y, Luo W, et al. A comprehensive empirical analysis of patch presence testing: Capabilities, limitations, and paths forward [J/OL]. *Proceedings of the ACM on Software Engineering*, 2026, 3(ISSTA): ISSTA053. DOI: [10.1145/3832144](https://doi.org/10.1145/3832144).
- [56] Zhang X, et al. Understanding coding agents for patch presence testing [Z]. 2026.
- [57] Wang X, Hu R, Gao C, et al. ReposVul: A repository-level high-quality vulnerability dataset [C/OL]//*Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Industry Challenge Track)*. 2024. DOI: [10.1145/3639478.3647634](https://doi.org/10.1145/3639478.3647634).
- [58] Cormack G V, Clarke C L A, Buettcher S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods [C]//*Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009: 758-759.
- [59] Zhou X, Kim K, Xu B, et al. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources [C/OL]//*Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 2024: 1-13. DOI: [10.1145/3597503.3639222](https://doi.org/10.1145/3597503.3639222).